

# Energy-Aware Task Distribution in Mobile UAV Fog Networks: A Tutorial-Grade Specification of a Hybrid MCDM + Attention-PPO Pipeline

From Physical-Layer and Mobility Models to Reinforcement-Learning  
Routing — Revised to the v4.3 Constants Audit and the 3-D Mobility Design

## Table of Contents

### Abstract

Unmanned Aerial Vehicles (UAVs) operating as flying “fog” servers can bring computation close to ground-based Internet-of-Things (IoT) devices, cutting the latency that would otherwise be incurred by sending data to a distant cloud. The central difficulty is that each UAV is a battery-limited, failure-prone, heterogeneous computer, and tasks arrive faster than any single drone can absorb. This paper presents a complete, reproducible specification of a two-stage decision system that decides which drone should coordinate the swarm and which pair of drones should execute each task, with the explicit goal of minimising the fraction of tasks that miss their deadlines (the drop rate) under hard energy and reliability constraints. Stage one is a multi-criteria decision-making (MCDM) controller that elects a “Master Fog Head” through a pipeline of four classical estimators: MEREC (objective criterion weighting), a Kalman filter (temporal smoothing of those weights), a Kullback–Leibler gate (a cheap test that avoids recomputation when nothing has changed), and SPOTIS (a rank-reversal-free ranking method). Stage two is a deep reinforcement-learning (RL) router built from a permutation-invariant self-attention encoder feeding an Actor–Critic network trained with Proximal Policy Optimisation (PPO); it emits a primary–backup pair of drones per task to provide fault tolerance. Relative to the earlier hover-only specification, this revision (i) makes the swarm *mobile* — UAVs fly in three dimensions under a demand-following controller, and their flight energy is charged against the same rotary-wing power curve that previously priced hovering; (ii) replaces the fixed a-priori tier table with an *emergent* tiering pipeline in which each drone’s intrinsic hardware determines its capability class; (iii) replaces the processing-capability criterion with a live *memory-efficiency* criterion; (iv) restores the system-reliability aggregate to its exact product form, accumulated in log space so it cannot underflow; and (v) re-derives every physical constant from a systematic paper-verified constants audit, so that each number is either independently confirmed against a reference paper or explicitly declared a documented assumption. We derive every physical model (delay, energy, mobility, memory, workload, reliability) from first principles, define every symbol exactly once, and motivate

every design choice. The paper is written so that a reader with no prior exposure to fog computing, multi-criteria decision-making, or reinforcement learning can follow the system end to end and re-implement it without further clarification.

**Keywords:** UAV fog computing; task offloading; UAV mobility; multi-criteria decision making; Kalman filtering; self-attention; Proximal Policy Optimisation; energy-aware scheduling; fault tolerance.

**Revision summary (audit rev v4.3 + mobility design).** This version of the paper aligns the specification with the current codebase, the v4.3 constants audit, and the 3-D mobility design document. The substantive changes are: (1) UAVs now move — three movement modes (`event_driven`, `random`, `static`), a fixed-altitude ring-and-layer station-keeping scheme, and flight energy charged from the full rotary-wing power curve  $P(V)$  (Section 4.5, 5.2); (2) 3-D slant range replaces planar distance in the channel model; (3) the clock envelope is [1.5, 3.0] GHz, exactly matching the verified reference value; (4) task computational intensity is a per-task random draw  $c_i \sim U[1600, 1800]$  cycles/byte instead of a fixed 1,500; (5) the noise floor is the pure thermal value  $N = kTB \approx 1.2013 \times 10^{-13}$  W (receiver noise figure 0 dB); (6) the channel-gain constants are  $v_1 \approx 1.42 \times 10^{-4}$ ,  $v_2 = 2.3$  (line-of-sight air-to-ground), replacing the earlier placeholder pair (1.0, 2.0); (7) tiers are emergent (two tiers of ten, ranked by computational efficiency) rather than assigned a priori; (8) the Stage-1 capability criterion is memory efficiency (ME), replacing processing capability (PC); (9) system reliability is the exact product aggregate of the reference reliability model, accumulated in log space; and (10) a constants-provenance section records which numbers are paper-verified and which are documented assumptions.

## 1 Introduction and Reading Guide

### 1.1 The problem in one paragraph

Imagine a field of two hundred ground sensors and terminals — soil monitors, cameras, hand-held units — each periodically producing a chunk of data that must be processed within a strict time budget (a deadline). A fleet of twenty battery-powered UAVs flies above them, each carrying a small onboard computer. A task can be sent up to a UAV, computed there, and the result returned, all far faster than a round-trip to a distant data centre. But the drones are not identical (some are fast, some slow), their batteries drain continuously while flying or hovering, and any drone may suffer a transient computation or radio failure. Because the drones now *move*, the geometry of the problem is itself dynamic: a drone repositioning over a demand hotspot shortens its radio links and speeds up every task it serves, while a drone loitering far from demand wastes both time and propulsion energy. The system must continuously answer two questions: (i) which drone should act as the coordinator that makes assignment decisions for the swarm, and (ii) for each incoming

task, which drone should execute it and which second drone should stand by as a backup. A wrong answer wastes energy, overloads a node, or — worst of all — lets a task miss its deadline. The metric we ultimately minimise is the *drop rate*: the fraction of tasks that are never completed on time.

## 1.2 Why this is hard, and why two stages

A natural instinct is to solve this with a single optimiser. Two facts make that impractical. First, the environment is non-stationary and partially random: batteries deplete, queues grow and shrink, drones physically relocate and their link qualities change with them, and nodes fail according to probabilistic rates, so the “best” decision at one instant is wrong a few seconds later. Second, the decision space is combinatorial: choosing an ordered (primary, backup) pair from  $N$  drones is an  $O(N^2)$  choice made thousands of times per episode. We therefore split the problem along its natural seam:

- **Stage 1 (slow, interpretable):** a lightweight statistical controller continually re-elects a single *Master Fog Head* — the drone best positioned right now to act as the swarm’s broker — using transparent, auditable multi-criteria reasoning. This stage answers “who decides.”
- **Stage 2 (fast, learned):** the elected broker runs a trained neural policy that, for each task, picks the executing pair. This stage answers “who executes.” It is learned rather than hand-coded because the trade-offs between speed, energy, load, position, and reliability are too subtle and too dynamic to capture in fixed rules.

## 1.3 How to read this paper

The paper is deliberately self-contained. Section 3 fixes notation so that no symbol ever means two things. Section 4 states the exact simulation environment, including the new mobility model and a provenance record for every constant. Section 5 derives the six physical models that turn raw configuration into the numbers every later stage consumes; these are pure engineering and require no machine-learning background. Section 6 explains Stage 1 (broker selection) one estimator at a time, each introduced from scratch. Section 7 is the heart of the paper: a ground-up account of the reinforcement-learning router, beginning with what reinforcement learning even is and ending with the exact training loss. Section 8 specifies how the chosen pairs are actually dispatched and how success and drop are scored. Section 9 is an implementation checklist. Throughout, three kinds of highlighted notes recur:

**Intuition** notes translate a formula into plain language before the mathematics arrives. A reader can follow the entire argument from these alone.

**Output** notes state precisely what each step emits — its shape, units, and range — so the data flowing between stages is never ambiguous.

**Design note** entries record a non-obvious modelling decision and the failure it prevents. These are where the paper's engineering judgement lives.

## 1.4 Contributions

The specific contributions of this work are: (1) a fully disambiguated, end-to-end model of an energy-aware *mobile* UAV fog system in which every physical quantity feeding the decision layers is derived rather than assumed, and in which every numeric constant carries an explicit provenance (paper-verified, literature-grounded, or documented assumption); (2) a 3-D mobility and flight-energy model that reuses the same rotary-wing power curve for hovering and cruising, together with a collision-free ring-and-altitude station-keeping scheme and three movement modes for controlled ablation; (3) an emergent hardware-to-tier pipeline in which capability classes are computed from per-node hardware rather than assigned a priori; (4) a four-stage MCDM broker-election pipeline (MEREC  $\rightarrow$  Kalman  $\rightarrow$  KL-gate  $\rightarrow$  SPOTIS) that is both responsive to change and computationally frugal, operating on a criteria set whose capability member is a live memory-efficiency signal; (5) an attention-based PPO router that is invariant to the number and ordering of live drones and that emits fault-tolerant primary-backup pairs through autoregressive decoding; and (6) a tutorial-grade exposition that makes each of the above re-implementable by a reader new to the field.

## 2 System Architecture at a Glance

Before any equations, it helps to see the whole pipeline as a single flow and to understand what each block hands to the next. Read it left to right: at the top of every 100 ms control tick, the *mobility layer* moves each UAV toward its station and drains the corresponding propulsion energy; the refreshed 3-D geometry then feeds the physical models, which compute the per-node quantities that describe the current health and position of the swarm. These feed Stage 1, which elects a broker. The broker runs Stage 2, the learned router, which emits a primary-backup pair per task. The execution layer dispatches the pair and reports whether the task succeeded, closing the loop by updating the telemetry that the models read on the next step.

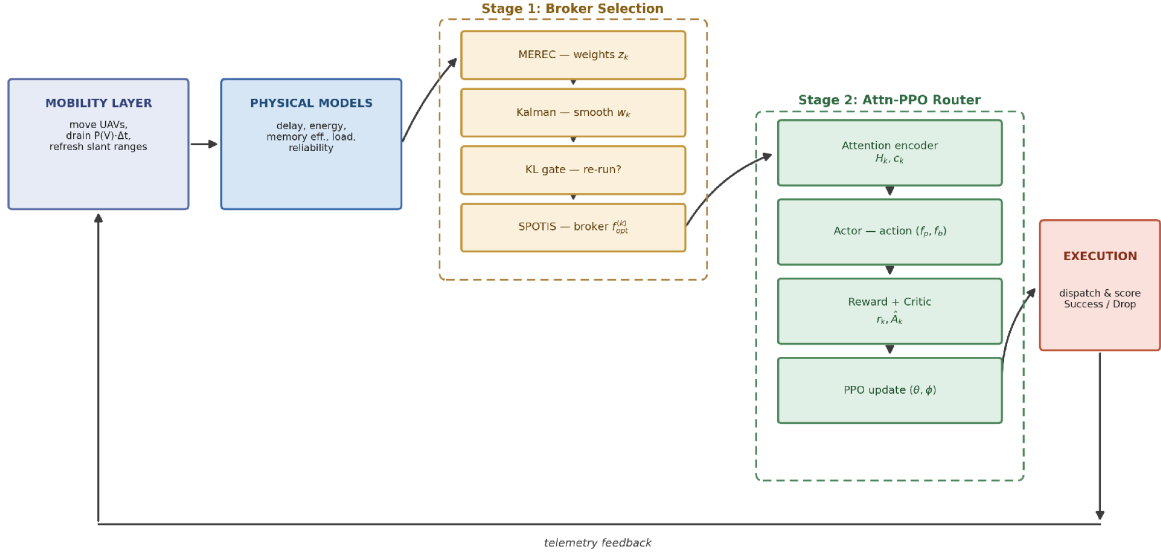


Figure 1: End-to-end pipeline. The mobility layer (new in this revision) repositions the swarm and charges propulsion energy each tick. The physical models convert live telemetry and geometry into per-node features. Stage 1 elects the broker through four cheap estimators. Stage 2 is the learned router run by that broker; it produces a fault-tolerant primary-backup pair which the execution layer dispatches and scores. The scored outcome updates the telemetry, closing the loop.

**The one-sentence mental model.** The mobility layer decides where each drone *is*; the physical models say how healthy each drone *is there*; Stage 1 picks the healthiest coordinator; Stage 2, run by that coordinator, learns the best executing pair for each task; and the execution layer reports whether it worked, feeding the result back so the next decision is better informed.

### 3 Master Notation

A recurring source of confusion in systems papers is symbol reuse — the same letter standing for two different quantities in two different equations. Table 1 fixes a single meaning for every symbol used in this paper; nothing outside this table is introduced silently later. The disambiguation guarantees beneath the table are worth reading once: they explain the specific collisions that the notation was designed to avoid.

Table 1: Master notation. Every symbol has exactly one meaning.

| Symbol                  | Meaning                                                                      | Type / Range      |
|-------------------------|------------------------------------------------------------------------------|-------------------|
| $I, F, T$               | Sets of IoT devices, fog UAVs, tasks                                         | sets              |
| $i, j$                  | Index of IoT device / fog node                                               | integer           |
| $\tau_i, s_i, d_i, l_i$ | Task $i$ : task object, payload size, deadline, CPU cycles                   | —, KB, ms, cycles |
| $c_i$                   | Task $i$ 's computational intensity, drawn $U[1600, 1800]$                   | cycles/byte       |
| $B$                     | Broker (Master Fog Head)                                                     | node              |
| $q_j$                   | Latent hardware quality of node $j$ (drives all specs)                       | $[0, 1]$          |
| $fr_j^{avg}$            | Node $j$ 's mean clock frequency (per-node, from $q_j$ )                     | GHz               |
| $CE_j^{eff}$            | Computational efficiency<br>$= fr_j / (\varsigma_j C_j 10^6)$ ; tiering only | dimensionless     |
| $g_{B,f_j}(t)$          | Time-varying channel power gain                                              | dimensionless     |
| $TR_{B,f_j}(t)$         | Broker/IoT $\leftrightarrow$ node $j$ data rate                              | bps               |
| $T^u, T^e, T^Q, D_i$    | Transmission, execution, queue, total delay of $\tau_i$                      | ms                |
| $Z_{B,f_j}$             | 3-D slant range between endpoints (incl. altitude)                           | m                 |
| $z_j$                   | Node $j$ 's fixed cruise altitude (assigned layer)                           | m, $[80, 150]$    |

| Symbol                                    | Meaning                                                     | Type / Range  |
|-------------------------------------------|-------------------------------------------------------------|---------------|
| $V$                                       | UAV forward (horizontal) speed                              | m/s           |
| $P(V)$                                    | Rotary-wing propulsion power at speed $V$                   | W             |
| $E_{initial,j}$                           | Initial battery energy of node $j$                          | J             |
| $E_{f_j}^{res}(t)$                        | Residual energy of node $j$                                 | J             |
| $SOC_j(k)$                                | State of Charge<br>$= E_{f_j}^{res} / E_{initial,j}$        | [0, 1]        |
| $P_{hover}, P_{proc}, P_{comm}, P_{idle}$ | Power draws (hover, processing, communication, idle)        | W             |
| $W_{f_j}$                                 | Per-node workload (queued CPU cycles)                       | cycles        |
| $WL$                                      | System workload imbalance<br>$= \sum_j  W_{f_j} - \bar{W} $ | cycles        |
| $ME_{f_j}$                                | Memory efficiency (free RAM + storage, normalised)          | [0, 1]        |
| $M_j^{P,tot}, M_j^{S,tot}$                | Node $j$ 's total RAM / storage                             | GB            |
| $M_j^{P,occ}, M_j^{S,occ}$                | Node $j$ 's occupied RAM / storage (from resident tasks)    | GB            |
| $C_j$                                     | Per-node MIPS capacity                                      | MIPS          |
| $\varsigma_j$                             | Per-node compute weight (lower = more capable)              | dimensionless |
| $\lambda_j, \mu_j$                        | Node $j$ computation / link failure rates                   | $s^{-1}$      |

| Symbol                                          | Meaning                                                                            | Type / Range           |
|-------------------------------------------------|------------------------------------------------------------------------------------|------------------------|
| $R_{i,j}^0$                                     | Base reliability of $\tau_i$ on node $j$                                           | $[0, 1]$               |
| $R_i$                                           | Primary-backup reliability of $\tau_i$                                             | $[0, 1]$               |
| $\log_{10} R_{sys}$                             | System reliability<br>aggregate = $\sum_i \log_{10} R_i$<br>(product in log space) | $\leq 0$               |
| $\bar{R}$                                       | Arithmetic mean reliability (secondary operator metric only)                       | $[0, 1]$               |
| $w_k$                                           | MCDM criteria weight vector at stage $k$                                           | $\Delta^4$ (sums to 1) |
| $P^{KF}, \Sigma_R^{KF}, Q^{KF}, K_k$            | Kalman error-cov., meas.-noise, process-noise, gain                                | matrices               |
| $\theta_{KL}$                                   | KL re-trigger threshold                                                            | scalar                 |
| $f_{opt}^{(k)}$                                 | Selected Master Fog Head at stage $k$                                              | node                   |
| $X_k$                                           | State: set of node feature vectors at stage $k$                                    | set of $R^5$           |
| $h_j, c_k$                                      | Attention node embedding / global context                                          | $R^{d_{model}}$        |
| $\pi_\theta, V_\phi$                            | Actor (policy) / Critic (value) networks                                           | neural nets            |
| $f_p, f_b$                                      | Primary / backup node of an action                                                 | node                   |
| $r_k, \hat{A}_k, V_{target}$                    | Reward, GAE advantage, value target                                                | scalar                 |
| $\gamma, \lambda, \varepsilon_{clip}, c_1, c_2$ | Discount, GAE $\lambda$ , PPO clip, value/entropy coeff.                           | scalars                |

| Symbol                       | Meaning                                              | Type / Range   |
|------------------------------|------------------------------------------------------|----------------|
| $M$                          | Critical-failure reward penalty                      | scalar (= 100) |
| $W_{uav}, \sigma_s, R_{rot}$ | UAV weight, rotor solidity, rotor radius (aero only) | N, -, m        |

**Disambiguation guarantees.** The letter  $R$  is never used alone for a model quantity: reliability is  $R_{ij}^0 / R_i / \log_{10} R_{sys} / \bar{R}$ , residual energy is  $E^{res}$ , rotor radius is  $R_{rot}$ , Kalman measurement noise is  $\Sigma_R^{KF}$ . The letter  $\lambda$  alone is the GAE parameter; failure rates carry a node subscript  $\lambda_j$ .  $M$  alone is the penalty; MIPS capacity is  $C_j$ .  $W_{f_j}$  is per-node load,  $W_{uav}$  is aircraft weight, and  $WL$  is the system-wide imbalance — three distinct objects, three distinct names. Finally,  $z_j$  is an altitude and  $z_{j,k}$  never appears; the MEREC measurement vector is written  $\mathbf{z}_k$  in bold to keep it distinct.

## 4 Simulation Environment Configuration

This section pins down the exact world the system operates in. Every number here is a fixed input; nothing is learned. A reader re-implementing the system can copy these tables verbatim. New to this revision, Section 4.6 records the *provenance* of every constant — which values are independently verified against the reference papers, which are literature-grounded calibrations, and which are documented code-level assumptions.

### 4.1 Global constraints

| Parameter                                  | Value                                          |
|--------------------------------------------|------------------------------------------------|
| Total tasks per episode                    | 2,500                                          |
| Grid                                       | 5,000 m $\times$ 5,000 m                       |
| Fog UAVs $ F $                             | 20                                             |
| IoT devices $ I $                          | 200                                            |
| IoT hotspots (Gaussian-mixture components) | 4 (spread $\sigma = 400$ m)                    |
| Task arrival                               | Poisson, 50 tasks/s (mean inter-arrival 20 ms) |
| Control tick                               | 100 ms (10 Hz)                                 |

| Parameter   | Value                                  |
|-------------|----------------------------------------|
| Global seed | 42 (all RNGs; runs fully reproducible) |

## 4.2 UAV hardware and emergent tiers

**Design note: tiers are computed, not assigned.** The earlier specification fixed a three-tier table a priori (4 + 10 + 6 drones with per-tier hardware) and gave each drone its tier’s hardware. The current design *inverts* that pipeline: hardware → computational efficiency → rank → tier. Each drone receives its own intrinsic hardware, its computational efficiency is computed from that hardware, the swarm is ranked, and the tier label is assigned by rank position. “Tier” is thus an *emergent* capability class rather than a fixed input, while preserving the original narrative — Tier-1 drones are the most capable all-round, so a large urgent task may only be serviceable by Tier 1.

The construction has four steps, executed once per episode:

1. **Sample quality.** Each UAV  $j$  draws a latent hardware quality  $q_j \sim U[0, 1]$ .
2. **Pin the envelope.** The 20 sampled values are affinely rescaled so the empirical minimum maps to 0 and the maximum to 1. This *pins* the hardware envelope endpoints every episode — there is always exactly one node at the strongest spec and one at the weakest — which keeps the task-feasibility guarantee of Section 4.4 exact.
3. **Interpolate hardware.** Every spec interpolates linearly in  $q_j$  between a WEAK endpoint ( $q = 0$ ) and a STRONG endpoint ( $q = 1$ ). All specs are monotone in  $q$ , so a higher-quality drone is better all-round: faster clock, more memory, bigger battery, lower failure rates.
4. **Rank and slice.** Compute each node’s computational efficiency

$$CE_j^{eff} = \frac{fr_j [Hz]}{s_j c_j \cdot 10^6},$$

rank the swarm in *descending*  $CE^{eff}$ , and assign tiers by rank position: the top 10 become **Tier 1**, the remaining 10 become **Tier 2** (tier counts [10, 10]).  $CE^{eff}$  is used *only* to form tiers; it is not a Stage-1 criterion (Section 5.3 explains what replaced it).

Table 2: Hardware envelope endpoints. Each node’s specs interpolate linearly between the WEAK and STRONG columns according to its rescaled quality  $q_j$ .

| Spec                  | WEAK ( $q = 0$ ) | STRONG ( $q = 1$ ) | Unit |
|-----------------------|------------------|--------------------|------|
| Mean clock $fr^{avg}$ | 1.5              | 3.0                | GHz  |

| Spec                                                | WEAK ( $q = 0$ ) | STRONG ( $q = 1$ ) | Unit     |
|-----------------------------------------------------|------------------|--------------------|----------|
| MIPS capacity $C$                                   | 12,000           | 24,000             | MIPS     |
| Compute weight $\varsigma$ (lower = better)         | 6                | 2                  | —        |
| Primary memory $M^{p,tot}$ (RAM)                    | 2.0              | 8.0                | GB       |
| Secondary storage $M^{s,tot}$                       | 4.0              | 8.0                | GB       |
| Initial battery $E_{init}$                          | 162,000 (~45 Wh) | 234,000 (~65 Wh)   | J        |
| Computation-failure rate $\lambda$ (lower = better) | 0.025            | 0.005              | $s^{-1}$ |
| Link-failure rate $\mu$ (lower = better)            | 0.010            | 0.001              | $s^{-1}$ |

The clock envelope [1.5, 3.0] GHz is an exact match to the verified reference value for fog-node CPU frequency (see Section 4.6); because the affine rescaling pins both endpoints, the cited bounds are *realised* in every episode, not merely spanned. Batteries correspond to 6S LiPo packs in the 45–65 Wh class. This heterogeneity is the entire reason routing is non-trivial — a large, compute-intense task may only be serviceable by the fastest drones, so those scarce nodes must not be squandered on tasks a weak node could have handled.

### 4.3 IoT devices

| Group                | Count | Transmit power $p$ |
|----------------------|-------|--------------------|
| Low-power sensors    | 150   | 2.51 W (34 dBm)    |
| High-power terminals | 50    | 31.62 W (45 dBm)   |

The two transmit-power classes matter because transmit power enters the channel data-rate formula (Section 5.1): a high-power terminal can push the same payload to a drone faster, lowering its transmission delay. The class boundaries themselves are verified reference values (34 dBm is near the LoRa regulatory ceiling; 45 dBm matches industrial 4G CPE uplink power). Spatially the 200 devices are drawn from a 4-component Gaussian mixture (cluster spread 400 m, centres kept 500 m inside the grid edge), creating four workload *hotspots* rather than a uniform spread — this is what makes both load balancing and demand-following mobility (Section 4.5) live concerns.

#### 4.4 Task generation (feasibility-guaranteed)

Tasks arrive as a Poisson process (random arrivals at a constant average rate of 50 tasks/s). Each task's payload size  $s_i$  is drawn from a bounded Pareto distribution with shape  $\alpha = 1.5$  (heavy-tailed: many small tasks, a few large ones), rejection-sampled into [60, 800] KB. The required CPU work scales linearly with size, but — new in this revision — the *computational intensity* is itself a per-task random variable, modelling a heterogeneous application mix:

$$l_i = s_i \cdot 1024 \cdot c_i \text{ cycles}, \quad c_i \sim U[1600, 1800] \text{ cycles/byte} (= [200, 225] \text{ cycles/bit}).$$

The intensity range is a strict subrange of the verified reference range of [200, 1400] cycles/bit; the upper truncation at 225 cycles/bit is a documented *deadline-feasibility ceiling*: the largest task (800 KB) on the fastest node (3.0 GHz) needs at most  $\approx 229$  cycles/bit to meet its deadline with the overhead budget below, so sampling above that would create tasks the whole swarm must drop. The nominal midpoint (1,700 cycles/byte) is used only for deterministic *representative* tasks inside the decision layers (Sections 6 and 7.3), never for real workload.

Deadlines depend on size class:

| Class                               | Deadline $d_i$ |
|-------------------------------------|----------------|
| Small task, $s_i \in [60, 250)$ KB  | 300 ms         |
| Large task, $s_i \in [250, 800]$ KB | 550 ms         |

**Why these deadlines (and why feasibility is guaranteed).** Larger tasks get a longer deadline, and the numbers are chosen so that no task is universally impossible while routing pressure is preserved. Treating execution time  $T^e = l_i / fr_j^{avg}$  as the dominant cost and reserving a fixed overhead budget  $\Omega = 50$  ms for transmission, broker decision, and light queuing, the largest serviceable payload on a node of clock  $fr$  at worst-case intensity ( $c_i = 1800$  cycles/byte) is  $s^{max} = (d_i - \Omega) fr / (1024 \cdot 1800)$ . Evaluating this across the pinned clock envelope gives:

| Node clock             | Small class (250 ms usable) | Large class (500 ms usable) |
|------------------------|-----------------------------|-----------------------------|
| Strongest, 3.0 GHz     | $\leq 407$ KB               | $\leq 814$ KB               |
| Mid-envelope, 2.25 GHz | $\leq 305$ KB               | $\leq 610$ KB               |
| Weakest, 1.5 GHz       | $\leq 203$ KB               | $\leq 407$ KB               |

Because the affine pinning guarantees a 3.0 GHz node in *every* episode, and  $814 \geq 800$  KB, every generated task is serviceable by at least one node — the very largest, most compute-intense task (800 KB at 1800 cycles/byte,  $l_i \approx 1.47 \times 10^9$  cycles) finishes on the strongest node in  $\approx 492$  ms of its 500 ms usable budget. At the other extreme, small tasks up to  $\approx 203$  KB are feasible on *every* node even at worst-case intensity (up to  $\approx 229$  KB at best-case intensity). Everything in between requires a progressively faster node, so the largest and densest tasks compete for the scarcest fast drones — which is precisely what makes the primary-backup routing mechanism essential, without ever creating a task the whole swarm must drop.

**Output (configuration).** A fully specified, internally consistent environment in which every generated task is serviceable by at least one node.

## 4.5 Mobility model (new)

Previously the UAVs were placed at random positions and never moved — they only ever hovered. This revision makes the swarm *fly*: where UAVs go, how they spread out without colliding, and how their flight energy is charged, all grounded in the demand-following repositioning treatment of the reference fog-UAV platforms and priced by the same rotary-wing power curve already used for hover.

### Design decisions at a glance.

| Design choice          | What it means                                                                             |
|------------------------|-------------------------------------------------------------------------------------------|
| Horizontal motion only | UAVs fly at a fixed altitude; the rotary-wing power curve $P(V)$ governs propulsion power |
| Fixed altitude $z_j$   | $z_j$ appears in the 3-D slant range but never changes (no climb energy to model)         |
| event_driven (default) | UAVs track their assigned IoT demand hotspot as it slowly drifts                          |
| random (ablation)      | Random-waypoint motion; a demand-agnostic baseline                                        |
| static (baseline)      | No motion; reproduces the original fixed-position simulation byte-for-byte                |

| Design choice          | What it means                                                                         |
|------------------------|---------------------------------------------------------------------------------------|
| Ring + altitude layers | Co-assigned UAVs are separated in $x$ – $y$ and in altitude — no collisions by design |

**Station-keeping without collisions.** Five UAVs are assigned to each of the four IoT hotspots ( $20/4 = 5$ ). They do not stack on the hotspot centroid: each takes a distinct slot on a **250 m standoff ring** around the demand cluster (so together they blanket it), and a distinct **altitude layer in the 80–150 m band**. Even if two stations momentarily overlapped horizontally, they remain separated vertically — so collisions cannot occur and no avoidance controller is needed, matching how the reference platforms treat UAVs as repositioning points rather than kinematic vehicles.

**Movement in the control loop.** The 100 ms control tick is unchanged in shape; mobility adds two steps at its top: (1) drift the demand model and move each UAV toward its current station at cruise speed, draining  $P(V) \times 0.1$  s of propulsion energy at its instantaneous speed  $V$ ; (2) recompute the 3-D geometry so the new positions feed every downstream calculation. Routing, reward, and reporting are exactly as before. The cruise speed is **12 m/s**, chosen deliberately: on the rotary-wing power curve (Section 5.2.1), cruising near 9–12 m/s actually costs *less* than hovering — the induced-power dip — and power rises steeply only above roughly 20 m/s.

**How movement propagates through the pipeline.** Moving a UAV changes its 3-D slant range to the ground devices. That single change ripples outward: data rate, transmission delay, link reliability, and communication energy all shift, which is what the MCDM broker and the PPO router then act on. Separately, the UAV’s speed sets its propulsion power and drains its battery. Mode selection is a single configuration switch (`MOVE_MODE`  $\in$  {`event_driven`, `random`, `static`}, also settable via environment variable), so the mobility ablation requires no code changes.

**What the modes look like, and what they cost.** Over a representative evaluation episode ( $\sim 50$  s, seed 42, untrained policy — the *mechanism*, not policy quality, is what these numbers illustrate): `event_driven` UAVs settle at a mean distance of  $\approx 0.23$  km from their nearest demand hotspot — essentially the ring radius — and stay there; `random` UAVs wander  $\approx 1.5$  km from demand throughout. Closer to demand means shorter links, lower delay, and lower communication energy. On the propulsion side, `event_driven` power starts at the cheap cruise level ( $\approx 788$  W) and climbs toward the hover cost ( $\approx 871$  W) as UAVs arrive on station; `random` stays at the  $\approx 779$  W cruise level because it never stops moving. Total propulsion energy is therefore *highest* for static hover ( $\approx 860$  kJ) and *lowest* for continuous cruise ( $\approx 770$  kJ) — a direct, visible consequence of the power curve.

**Design note: motion is not just a cost.** Enabling realistic movement did not merely add an energy expense — it exposed a genuine energy trade-off the routing agent can exploit

(cruising is cheaper than hovering), while the static mode still reproduces the original fixed-position numbers exactly, so before/after comparisons are clean.

## 4.6 Constant provenance (audit summary)

Every numeric constant in this specification was audited against the project’s reference papers. Of the 49 constants examined, the breakdown is: 6 verified exact, 4 verified in functional form (1 form-only, 3 form-and-exact), 2 verified as method sources only, 1 verified in-range, 3 roughly consistent, 2 inconsistent with the nearest stated paper value (flagged and resolved), 3 not applicable (features not modelled), and 28 documented assumptions. The load-bearing findings:

- **Verified exact:** the  $5 \times 5$  km grid; the 30 MHz channel bandwidth; the 34/45 dBm IoT transmit-power classes; the [60, 800] KB payload range; the thermal noise floor ( $-174$  dBm/Hz, noise figure 0 dB); and the [1.5, 3.0] GHz fog-CPU clock envelope.
- **Verified in functional form:** the Shannon–Hartley rate with power-law channel gain  $g = v_1 d^{-v_2}$ ; the exponential base-reliability model  $R^0 = \exp(-\lambda l / fr - \mu s / TR)$  with the parallel primary-backup combination; the product system-reliability aggregate (restored in this revision, Section 5.5); and the workload-imbalance formula  $WL = \sum_j |W_{f_j} - \overline{W}|$ .
- **Verified in-range:** the computational intensity  $c_i \in [200, 225]$  cycles/bit, a strict subrange of the reference [200, 1400] cycles/bit, truncated at the documented feasibility ceiling.
- **Deliberately not adopted:** the reference paper’s own channel constants  $(v_1, v_2, N) = (10^{-3}, 4, 10^{-10} W)$  describe a short-range terrestrial link; on the verified 5 km grid they yield  $\approx 28$  bit/s at mid-cell (the smallest 60 KB task would take  $\approx 4.9$  h against a 300 ms deadline), collapsing the simulation. The adopted pair —  $v_1 = (\lambda_c / 4\pi)^2 \approx 1.42 \times 10^{-4}$  (Friis free-space reference gain at 1 m, carrier 2 GHz) and  $v_2 = 2.3$  (within the measured line-of-sight air-to-ground path-loss exponent range 2.0–2.5) — is a *literature-grounded assumption*, standard in air-to-ground channel modelling but not sourced from a project paper.
- **Internal inconsistency in a source, resolved:** the same reference gives fog-CPU frequency as [1.5, 3] GHz in its body text but  $50 \times 10^9$  cycles/s in its parameter table — over  $10\times$  higher. The table value would make worst-case compute  $\approx 25$  ms, erasing deadline pressure and collapsing the compute dimension of the decision problem; the body-text value is adopted, the table value rejected.
- **Documented assumptions:** the full hardware envelope (Table 2), every aerodynamic constant, the per-mode power draws, the broker sub-stage timings, the Kalman tuning triple, the KL threshold, the SPOTIS bound values, the queue-depth bound, the arrival rate, the Pareto shape, the deadline table, and every

attention/PPO hyperparameter. None of these appear in a project paper; each is recorded with its calibration rationale where one exists.

## 5 System Models: Turning Configuration into Live Telemetry

Everything downstream — both the broker election and the learned router — reasons over a handful of numbers that summarise each drone’s current condition. This section derives those numbers. There are six models, and each emits one family of quantities: **delay** (will the task finish in time?), **energy** (how much battery is left, including what flight itself costs?), **memory efficiency** (how much memory headroom is free?), **workload** (how busy and how balanced is the swarm?), and **reliability** (how likely is the assignment to survive failures?), plus the **tiering** quantity that classifies the hardware. None of this requires any machine-learning knowledge; it is classical queueing, communications, flight mechanics, and reliability engineering.

### 5.1 Delay model

The total time to service a task is the sum of four parts: getting the data to the drone (transmission), waiting behind other tasks (queuing), computing it (execution), and the small fixed cost of the broker’s decision. We build each piece, then add them.

#### 5.1.1 Transmission delay

A radio link’s speed depends on how strong the received signal is relative to noise. The channel power gain captures how much the signal weakens with the **3-D slant range**  $Z_{B,f_j}$  between the endpoints — the ground device (or broker) and the drone at its cruise altitude  $z_j$ :

$$g_{B,f_j}(t) = v_1 Z_{B,f_j}^{-v_2}, \quad Z_{B,f_j} = \sqrt{(\Delta x)^2 + (\Delta y)^2 + z_j^2} \quad (\geq 1 \text{ m}).$$

Because the drones move (Section 4.5),  $Z_{B,f_j}$  — and therefore everything downstream of it — is genuinely time-varying. The achievable data rate follows the Shannon–Hartley law — the maximum reliable bits per second a channel of bandwidth  $\omega$  can carry at a given signal-to-noise ratio — and the transmission delay is the payload *in bits* divided by that rate:

$$TR_{B,f_j}(t) = \omega \log_2 \left( 1 + \frac{g_{B,f_j}(t)p}{N} \right), \quad T_{f_j, \tau_i}^u = \frac{s_i \cdot 1024 \cdot 8}{TR_{B,f_j}(t)}.$$

Here  $p$  is the transmitter’s power (recall the two IoT power classes) and  $N$  is the noise power. The explicit  $1024 \cdot 8$  factor converts the KB payload to bits; leaving it implicit was a unit ambiguity in the earlier specification.

| Constant | $\omega$ | $N$                                | $v_1$                        | $v_2$                   | carrier $f_c$ |
|----------|----------|------------------------------------|------------------------------|-------------------------|---------------|
| Value    | 30 MHz   | $1.2013 \times 10^{-13} \text{ W}$ | $(\lambda_c/4\pi)^2 \approx$ | 2.3 (LoS air-to-ground) | 2 GHz         |

The noise floor is the pure Johnson–Nyquist thermal term  $N = kTB$  at  $T = 290$  K with a receiver noise figure of 0 dB — arithmetically identical to the verified — 174 dBm/Hz floor integrated over the 30 MHz band ( $\approx -99.2$  dBm). The resulting link budget gives an SNR of roughly 10 dB at the 2.5 km mid-cell and roughly 3 dB at the 5 km cell edge for a 2.51 W sensor — i.e. coverage holds across the whole grid, with a realistic 30–500 Mbps rate envelope.

**Intuition.** Doubling the payload doubles transmission time. Moving a drone farther away weakens the signal as  $Z^{2.3}$ , which lowers the rate and so raises the delay — and, since drones now fly, this is exactly the lever the event\_driven mobility mode pulls in the system’s favour by shortening slant ranges to demand. Higher transmit power  $p$  pushes data through faster. The logarithm means that once the signal is already strong, adding still more power buys little extra rate — a classic diminishing return.

### 5.1.2 Execution delay

Computing a task takes its required cycles divided by the drone’s clock speed:

$$T_{f_j \tau_i}^e = \frac{l_i}{f r_j^{avg}}.$$

Each node carries a single per-node mean clock  $f r_j^{avg}$  (interpolated from its quality  $q_j$ ; Section 4.2), and we always use it in every delay and energy formula — never a bare  $f r_j$  — so the model is unambiguous. The strongest node at 3.0 GHz finishes the same task in  $1.5/3.0 = 50\%$  of the time the weakest node at 1.5 GHz would need: this single ratio is why fast drones are worth conserving.

### 5.1.3 Queuing delay (dual-queue, earliest-deadline-first)

A drone rarely sits idle waiting for one task; it holds a queue. We use two queues per drone: a backup queue  $Q_j^B$  with absolute, non-preemptive priority over the primary queue  $Q_j^P$ , each internally ordered by Earliest-Deadline-First (EDF — the task with the nearest deadline runs next). The unified execution order is the EDF-sorted backup queue followed by the EDF-sorted primary queue. If  $n_j = |Q_j^B| + |Q_j^P|$  tasks sit in that merged order, the average queuing delay — the mean time a task waits while those ahead of it are processed — is

$$T_{f_j}^{Q,avg} = \frac{1}{n_j f_j^{avg}} \sum_{k=1}^{n_j-1} (n_j - k) l_k.$$

**Why this formula.** The  $k$ -th task in line must wait for every task ahead of it to finish. The first task makes  $(n_j - 1)$  later tasks wait for its  $l_1$  cycles, the second makes  $(n_j - 2)$  wait, and so on; summing those contributions and dividing by the clock (to convert cycles to time) and by  $n_j$  (to average over all positions) gives the expression above. Bottom line: a long queue of big tasks inflates everyone's wait. A per-node queue-depth design bound of 64 slots caps the worst case (Section 5.4).

#### 5.1.4 Broker decision overhead, and the total

Propagation time (the signal's flight time) is single-hop and microsecond-scale, negligible against millisecond execution, so it is omitted. The broker's own decision cost is a fixed, bounded per-stage overhead:

$$T_B^{(k)} = T_{MEREC} + T_{Kalman} + T_{KL} + (I_{trigger} \cdot T_{SPOTIS}) + T_{PPO\_infer},$$

where  $I_{trigger} \in \{0, 1\}$  is 1 only when the KL gate (Section 6.4) decides to re-run SPOTIS — so the most expensive sub-step is paid only when needed.

| $T_{MEREC}$ | $T_{Kalman}$ | $T_{KL}$ | $T_{SPOTIS}$                        | $T_{PPO\_infer}$ |
|-------------|--------------|----------|-------------------------------------|------------------|
| 0.50 ms     | 0.10 ms      | 0.05 ms  | 1.00 ms<br>(only when<br>triggered) | 0.80 ms          |

The total delay of task  $\tau_i$  routed to drone  $f_j$  is the sum of the four parts:

$$D_i = T_B^{(k)} + T_{f_j \tau_i}^u + T_{f_j}^{Q,avg} + T_{f_j \tau_i}^e.$$

**Output (delay).** A scalar  $D_i$  in milliseconds for each candidate assignment, directly comparable to the task's deadline  $d_i$ : the task is feasible on that drone iff  $D_i \leq d_i$ .

## 5.2 Propulsion, residual energy, and State of Charge

A flying drone burns power continuously, and computing or communicating burns more. The battery is the binding constraint of the whole system, so we track it precisely. New in this revision, propulsion is priced at the drone's *actual speed* on a single power curve, so hovering and cruising are two points of the same model with no discontinuity to reconcile.

### 5.2.1 The rotary-wing power curve (hover and flight on one model)

For a rotary-wing UAV at forward speed  $V$ , the propulsion power is the sum of blade-profile, induced, and parasite terms:

$$P(V) = P_0 \left( 1 + \frac{3V^2}{U_{tip}^2} \right) + P_i \left( \sqrt{1 + \frac{V^4}{4v_0^4}} - \frac{V^2}{2v_0^2} \right)^{1/2} + \frac{1}{2} d_0 \rho \sigma_s A V^3.$$

Hovering is simply the  $V = 0$  point of this curve, which collapses cleanly to

$$P_{hover} = P(0) = P_0 + P_i.$$

| $W_{uav}$ | $\rho$                  | $R_{rot}$ | $A = \tau U_{tip}$   | $\sigma_s$ | $d_0$ | $v_0$   | $P_0$  | $P_i$    | $P_{hover}$ |
|-----------|-------------------------|-----------|----------------------|------------|-------|---------|--------|----------|-------------|
| 100 N     | 1.225 kg/m <sup>3</sup> | 0.5 m     | 0.785 m <sup>2</sup> | 200 m/s    | 0.05  | 0.3 m/s | 79.8 W | 790.52 W | 870.52 W    |

Each control tick, every UAV is charged  $P(V) \times 0.1$  s of propulsion energy at its current speed:  $P(0) = 870.52$  W while station-keeping, and roughly 779–788 W at the 12 m/s cruise — the induced-power dip that makes moderate forward flight *cheaper* than hovering.

**Intuition.** The dominant cost is just staying airborne: at roughly 871 W of hover power, a drone spends far more energy holding position than it does computing (processing power is only 10 W). This is why wasted queue time is wasted battery — a task sitting in a queue still costs hover power every millisecond — and why the reward later explicitly penalises queue energy. The counter-intuitive-but-correct corollary of the power curve is that a *moving* swarm can spend less propulsion energy than a stationary one: over a representative episode, total propulsion energy is highest for static hover ( $\approx 860$  kJ) and lowest for continuous cruise ( $\approx 770$  kJ).

### 5.2.2 Computing and communication energy

For the set of tasks  $E_j$  assigned to drone  $j$ , the energy spent computing, communicating, and idling is

$$E_{proc,j} = P_{proc} \sum_{\tau_i \in E_j} \frac{l_i}{fr_j^{avg}}, \quad E_{comm,j} = 2P_{comm} \sum_{\tau_i \in E_j} \frac{s_i \cdot 1024 \cdot 8}{TR_{B,f_j}(t)}, \quad E_{idle,j} = P_{idle} t_{idle}.$$

The factor of 2 on communication accounts for both receiving the task and returning the result. Task energy is applied to a node only when it *actually* executes (the primary always; the backup only if it is dispatched — Section 8).

| $P_{proc}$ | $P_{comm}$ | $P_{idle}$ |
|------------|------------|------------|
| 10 W       | 2 W        | 3 W        |

### 5.2.3 Residual energy and State of Charge

The battery left at time  $t$  is the initial charge minus everything spent on flying, computing, communicating, and idling:

$$E_{f_j}^{res}(t) = E_{initial,j} - \int_0^t P(V(\tau)) d\tau - [E_{proc,j} + E_{comm,j} + E_{idle,j}],$$

clamped at zero so the normalisation below stays in range. Normalising by the battery's capacity gives the single most important scalar in the system, the State of Charge:

$$SOC_j(k) = \frac{E_{f_j}^{res}(t_k)}{E_{initial,j}} \in [0, 1].$$

**Output (energy).** Residual energy  $E_{f_j}^{res}$  in joules and  $SOC_j \in [0, 1]$  per drone. Every safety mask later in the paper (the 0.30 primary threshold, the 0.15 backup and Return-to-Home thresholds) and the energy term of the reward read this exact quantity.

## 5.3 Memory efficiency (the capability criterion) and computational efficiency (the tiering quantity)

The earlier specification blended free memory and raw compute into one “processing capability” score. The current design *separates* those two ideas, because they now do different jobs:

- **Computational efficiency**  $CE_j^{eff} = fr_j / (\zeta_j C_j 10^6)$  (higher = better) is a static property of the hardware. It is used *once per episode* to rank the swarm and form the tiers (Section 4.2) — and then never again. It is deliberately *not* a Stage-1 criterion: a static quantity cannot help the broker election react to live conditions, and its information survives as the tier feature the router sees (Section 7.3).
- **Memory efficiency**  $ME_{f_j}$  (higher = better) is the *live* capability criterion the broker sees. It is the weighted free memory across RAM and storage, normalised by the maximum attainable free memory (the strongest node with empty queues) and clipped to the unit interval:

$$ME_{f_j} = clip\left(\frac{\alpha(M_j^{P,tot} - M_j^{P,occ}) + \beta(M_j^{S,tot} - M_j^{S,occ})}{ME_{norm}}, 0, 1\right), \quad ME_{norm} = \alpha M^{P,strong} + \beta M^{S,strong} = 8.0 \text{ GB}.$$

| $\alpha$ | $\beta$ | RAM<br>footprint              | Storage<br>footprint          | RAM floor        | Storage<br>floor |
|----------|---------|-------------------------------|-------------------------------|------------------|------------------|
| 0.5      | 0.5     | $2.5 \times 10^{-4}$<br>GB/KB | $5.0 \times 10^{-4}$<br>GB/KB | 0.010<br>GB/task | 0.020<br>GB/task |

The occupancies  $M_j^{P,occ}, M_j^{S,occ}$  are recomputed every control tick from the tasks currently *resident* in the node's queues: each resident task occupies memory proportional to its payload (through the documented inflation factors above, modelling the decompressed working set and resident framework image of an isolated runtime) with a per-task runtime floor, and the total is clamped to the node's physical capacity. ME therefore captures **both** capacity (per-node  $M^{P,tot}/M^{S,tot}$  differ, so ME discriminates across nodes even at zero load) **and** occupancy (ME falls as a node fills up and rises as its queue drains). The footprint constants are calibrated so a busy node reaches roughly 10–20% RAM occupancy at realistic queue depths — a clear load-coupling signal that modulates ME without saturating against capacity.

**Design note: why ME replaced PC.** The old PC score min-max-normalised its components across the *live* swarm, so a node's score depended on which other nodes happened to be alive — the same instability SPOTIS was chosen to avoid. ME is normalised by a *fixed* constant ( $ME_{norm}$ ), so each node's score is independent of the rest of the swarm, keeping the whole Stage-1 pipeline rank-reversal-free end to end. And unlike the static compute half of PC, ME genuinely moves during an episode, giving MEREC a capability signal that actually varies.

**Output (capability).** A single scalar  $ME_{f_j} \in [0, 1]$  per drone, combining free-memory capacity and live occupancy — one of the five MCDM criteria.

## 5.4 Workload (per-node load and system imbalance)

Two distinct workload notions matter, and we name them distinctly. The per-node load is the total queued work on a drone; the system imbalance measures how unevenly that load is spread:

$$W_{f_j} = \sum_{p \in Q_j^P} l_p + \sum_{b \in Q_j^B} l_b, \quad \overline{W} = \frac{1}{|F|} \sum_j W_{f_j}, \quad WL = \sum_{j=1}^{|F|} |W_{f_j} - \overline{W}|.$$

A fixed design bound caps the queue: at most 64 tasks may be resident per node, so the worst-case per-node load is  $W^{max} = 800 \cdot 1024 \cdot 1800 \cdot 64 \approx 9.44 \times 10^{10}$  cycles (every

slot holding the largest, densest task). This bound is what makes the SPOTIS workload bound of Section 6.5 a true a-priori ceiling.

**Why imbalance, not total load?** The swarm’s total work is fixed by the arriving tasks — we cannot reduce it. What we can control is its distribution.  $WL$  is the sum of deviations from the average: it is zero when every drone carries an equal share and grows when some drones are swamped while others idle. Minimising  $WL$  spreads tasks so no single drone becomes a bottleneck whose queue blows past deadlines.

**Output (workload).** Per-node  $W_{f_j}$  in cycles (an input feature to the attention encoder) and the scalar system imbalance  $WL$  in cycles (an input to the reward). These are deliberately distinct objects with distinct names.

## 5.5 Reliability (heterogeneous primary–backup)

Any drone can suffer a transient computation failure (rate  $\lambda_j$ ) or a link failure (rate  $\mu_j$ ). Modelling these as exponential processes, the probability that a single assignment of  $\tau_i$  to drone  $j$  survives both long enough to finish is

$$R_{i,j}^0 = \exp\left(-\lambda_j \frac{l_i}{fr_j^{avg}} - \mu_j \frac{s_i \cdot 1024 \cdot 8}{TR_{B,f_j}(t)}\right).$$

This is exactly why we assign two drones. Because the primary  $f_p$  and backup  $f_b$  are different nodes (and so fail independently), the task succeeds if at least one survives — the standard parallel-reliability form:

$$R_i = 1 - \left(1 - R_{i,f_p}^0\right)\left(1 - R_{i,f_b}^0\right) = R_{i,f_p}^0 + R_{i,f_b}^0 - R_{i,f_p}^0 R_{i,f_b}^0.$$

**System reliability: the exact product, accumulated in log space.** The reference

reliability model defines system reliability as the *product* over all tasks,  $R_{sys} = \prod_i R_i$  — the

probability that *every* task succeeds. The earlier specification replaced this with an arithmetic mean, on the grounds that a product over 2,500 tasks at  $R_i \approx 0.95$  underflows

to  $\approx 4.7 \times 10^{-56}$  numerically. That substitution silently changed the metric’s semantics: a mean is forgiving of a single catastrophic task failure that the product correctly flags. This revision restores the product *exactly* by accumulating it in log space:

$$\log_{10} R_{sys} = \sum_{i=1}^{|T|} \log_{10} R_i,$$

which is mathematically identical to the product but *sums* instead of multiplying, so it never underflows and decreases linearly (not exponentially) with task count — the log of the value above is simply  $\approx -55.3$  and stays informative. This also mirrors how the reference model itself reports the metric: as a “number of nines” (a log transform), never the raw product. Two derived reporting forms accompany it:

$$R_{geo} = (\prod_i R_i)^{1/|T|} = 10^{\log_{10} R_{sys}/|T|}, \quad \text{nines} = -\log_{10}(1 - R_{sys}) \text{ (capped at 15)}.$$

A dropped task records  $R_i = 0$  exactly, whose log is  $-\infty$ ; each  $R_i$  is therefore clamped below at  $10^{-12}$  inside the aggregate, turning every outright failure into a finite, calibrated  $-12$  contribution to  $\log_{10} R_{sys}$  instead of a metric-destroying infinity. The arithmetic mean

$\bar{R} = \frac{1}{|T|} \sum_i R_i$  is retained *only* as a clearly-labelled secondary operator metric; it is explicitly not the system aggregate. The Stage-1 criterion and the reward continue to use per-task / per-node reliabilities, which live comfortably in  $[0, 1]$ .

**Why two independent drones beat one reliable drone.** If a single drone succeeds with probability 0.9, it fails one task in ten. Two independent drones at 0.9 each fail together only with probability  $0.1 \times 0.1 = 0.01$  — one task in a hundred. Redundancy turns a 10% failure into a 1% failure, which is the whole point of the backup. (It also explains the corrected backup mask in Section 7.4: a backup with a dead battery provides no redundancy.)

**Output (reliability).** Per-task  $R_i \in [0, 1]$ ; the episode-level aggregate  $\log_{10} R_{sys} \leq 0$  with its geometric-mean and number-of-nines reporting forms; and the secondary operator mean  $\bar{R} \in [0, 1]$ . The per-node base reliability (evaluated on a representative task at the node’s current link rate) is a beneficial MCDM criterion and an encoder feature.

## 5.6 The optimisation objective

With every quantity defined, the problem is stated cleanly: choose a primary–backup pair for each task so as to minimise the drop rate, subject to feasibility and a reliability floor.

$$\min_{\{(f_p, f_b)_i\}} \text{Drop\_Rate} \quad \text{s.t.} \quad \forall i: f_p \neq f_b, \quad SOC_{f_p} \geq 0.30, \quad SOC_{f_b} \geq 0.15, \quad R_i \geq R_{min},$$

with  $R_{min} = 0.95$ . In words: the two drones must differ; the primary must hold at least 30% charge and the backup at least 15%; and the redundant pair must meet a 95% reliability floor. Stage 1 and Stage 2 together produce assignments that respect these constraints while driving the drop rate down.

## 6 Stage 1 — Master Fog Head Selection

### 6.1 What this stage decides, and why it is not learned

The swarm needs a coordinator — a single drone, the Master Fog Head or broker, that runs the routing policy and dispatches tasks. The natural choice is whichever drone is currently in the best overall condition. “Best overall” must balance five competing criteria, three of which are beneficial (more is better: residual energy  $E^{res}$ , memory efficiency  $ME$ , reliability  $R^0$ ) and two non-beneficial (less is better: expected delay  $D$ , per-node workload  $W_{f_j}$ ).

Balancing competing criteria to rank alternatives is exactly the job of Multi-Criteria Decision Making (MCDM).

The fixed criteria order used everywhere in the pipeline is

$[E^{res}(+), ME(+), R^0(+), D(-), W(-)]$ . The delay and reliability columns are evaluated on a deterministic *representative* task — the payload-range midpoint of 430 KB at the nominal intensity of 1,700 cycles/byte, against each node’s mean distance to the IoT population at the population-weighted mean transmit power — so all candidates are compared on identical work.

We use a transparent statistical pipeline rather than a learned model here for two reasons. First, the choice of coordinator must be auditable: an operator should be able to see why a drone was elected. Second, this decision is cheap and frequent, so it must be fast. The pipeline has four steps, run each stage  $k$ :

$MEREC \xrightarrow{\sim \text{measure weights}} Kalman \xrightarrow{\sim \text{smooth weights}} KL \text{ gate} \xrightarrow{\sim \text{recompute?}} SPOTIS \xrightarrow{\sim \text{rank \& elect}} .$

The criteria start with equal weights  $w_0 = [0.2, 0.2, 0.2, 0.2, 0.2]^T$ .

**The four steps in plain language.** MEREC measures how important each criterion is right now, from the data itself. The Kalman filter smooths those measurements over time so the weights do not jitter from moment to moment. The KL gate is a cheap test: if the smoothed weights barely changed, skip the expensive ranking and keep the current broker. Otherwise SPOTIS ranks all drones against an ideal profile and elects the closest one.

### 6.2 MEREC — measuring criterion importance from data

MEREC (Method based on the Removal Effects of Criteria) answers: how much does each criterion actually matter for telling the drones apart right now? Its idea is the “removal effect.” For each criterion, it asks how much the overall assessment of the drones would change if that criterion were removed. A criterion whose removal barely changes anything is uninformative and gets a small weight; a criterion whose removal scrambles the picture

is decisive and gets a large weight. Writing  $\Delta_j^{rem}$  for the removal effect of criterion  $j$  over the normalised node values, the measurement weight is its share of the total effect:

$$z_{j,k} = \frac{\Delta_j^{rem}}{\sum_{c=1}^m \Delta_c^{rem}}.$$

**Output (MEREC).** A measurement weight vector  $z_k \in \Delta^4$  (five non-negative entries summing to 1) — the data-driven importance of each criterion at this instant.

### 6.3 Kalman filter — smoothing the weights over time

MEREC’s instantaneous weights are noisy: a momentary spike in one drone’s queue can make “delay” look briefly all-important. If we re-elected the broker on every twitch, the coordinator role would flap between drones. The Kalman filter is the standard tool for estimating a slowly-changing true value from a stream of noisy measurements; we apply it to the weight vector itself.

A Kalman filter alternates two steps. **Predict** carries the previous estimate forward and grows its uncertainty by the process noise  $Q^{KF}$  (how much the true weights might have drifted):

$$w_{k|k-1} = w_{k-1|k-1}, \quad P_{k|k-1}^{KF} = P_{k-1|k-1}^{KF} + Q^{KF}.$$

**Update** blends the prediction with the new MEREC measurement  $z_k$ , weighting them by the Kalman gain  $K_k$ , which is large when the measurement is trustworthy relative to our uncertainty and small otherwise:

$$K_k = P_{k|k-1}^{KF} (P_{k|k-1}^{KF} + \Sigma_R^{KF})^{-1},$$

$$w_{k|k} = w_{k|k-1} + K_k (z_k - w_{k|k-1}), \quad P_{k|k}^{KF} = (I - K_k) P_{k|k-1}^{KF}.$$

| $Q^{KF}$      | $\Sigma_R^{KF}$ | $P_0^{KF}$ |
|---------------|-----------------|------------|
| $10^{-4} I_5$ | $10^{-2} I_5$   | $I_5$      |

**The gain is a trust dial.** The term  $z_k - w_{k|k-1}$  is the “surprise” — how far the new measurement is from what we expected. The gain  $K_k$  decides how much of that surprise to believe. With measurement noise  $\Sigma_R^{KF} = 10^{-2}$  a hundred times larger than process noise

$Q^{KF} = 10^{-4}$ , the filter trusts its smooth internal estimate far more than any single jumpy measurement — exactly the damping we want.

**Output (Kalman).** Smoothed criterion weights  $w_k = w_{k|k}$ , renormalised to sum to 1. Temporal smoothing prevents the weight oscillation that would otherwise destabilise the broker election.

#### 6.4 KL gate — recompute only when something changed

SPOTIS (next) is the most expensive step. We should pay for it only when the criterion weights have genuinely shifted. The Kullback–Leibler divergence  $D_{KL}$  measures how different two probability distributions are; since each weight vector sums to 1 it is a distribution over criteria, so we compare this stage's weights to last stage's:

$$D_{KL}(w_k \parallel w_{k-1}) = \sum_{j=1}^m w_{j,k} \ln \frac{w_{j,k}}{w_{j,k-1}}.$$

The decision is a simple threshold on  $\theta_{KL} = 0.05$ :

- $D_{KL} \leq \theta_{KL}$ : weights essentially unchanged  $\Rightarrow$  keep the previous broker,  $f_{opt}^{(k)} = f_{opt}^{(k-1)}$ , and skip SPOTIS.
- $D_{KL} > \theta_{KL}$ : weights shifted meaningfully  $\Rightarrow$  re-run SPOTIS.

**Output (KL gate).** A boolean run\_SPOTIS. This is what makes the  $I_{trigger}$  term in the broker overhead  $T_B^{(k)}$  usually zero, saving the 1 ms SPOTIS cost on stable stages.

#### 6.5 SPOTIS — ranking and electing the broker

When recomputation is warranted, SPOTIS (Stable Preference Ordering Towards Ideal Solution) ranks the drones. It scores each candidate by its weighted, normalised distance from a fixed Ideal Solution Profile  $ISP$  (the best attainable value on every criterion), computed over the raw decision matrix clipped into fixed a-priori bounds; the closest candidate wins:

$$D_x^{SPOTIS} = \sum_{j=1}^m w_{j,k} \frac{|v_{x,j} - ISP_j|}{S_{max,j} - S_{min,j}}, \quad f_{opt}^{(k)} = \arg \min_{x \in F} D_x^{SPOTIS}.$$

The bounds are set a priori, independent of the live candidate set, with  $ISP_j = S_{max,j}$  for beneficial criteria and  $S_{min,j}$  for non-beneficial ones:

| Criterion     | $[S_{min}, S_{max}]$                      | Rationale                                             |
|---------------|-------------------------------------------|-------------------------------------------------------|
| $E^{res} (+)$ | $[0, 234, 000] \text{ J}$                 | 0 = depleted; 234 kJ = strongest node's full charge   |
| $ME (+)$      | $[0, 1]$                                  | unit interval by construction                         |
| $R^0 (+)$     | $[0, 1]$                                  | unit interval by construction                         |
| $D (-)$       | $[0, 550] \text{ ms}$                     | 0 = instant; 550 ms = worst allowed deadline          |
| $W (-)$       | $[0, 9.44 \times 10^{10}] \text{ cycles}$ | empty queue ... 64 slots of the largest, densest task |

**Why SPOTIS and not a simpler ranker.** Many MCDM methods suffer *rank reversal*: adding or removing an irrelevant candidate can flip the order of the others — clearly undesirable when drones join or die constantly. SPOTIS compares every candidate to a fixed external ideal rather than to the other candidates, so the ranking of any two drones never depends on a third. That stability is precisely what a swarm with a changing membership needs — and it is why the bounds above are constants, never live minima/maxima, and why the ME criterion is normalised by a fixed constant rather than across the live swarm (Section 5.3).

**Output (SPOTIS).** The elected Master Fog Head  $f_{opt}^{(k)}$  plus a sorted failover list  $L^{(k)}$  (used if the elected broker itself fails).

## 7 Stage 2 — Energy-Aware Routing via Attention-PPO

This is the heart of the system: the learned router that the elected broker runs. We build it from the ground up. Section 7.1 explains what reinforcement learning is for a reader who has never seen it. Section 7.2 justifies the two key architectural choices (attention and PPO). The remaining subsections specify the encoder (7.3), the action mechanism (7.4), the reward (7.5), the critic and advantage estimator (7.6), and the training loss (7.7).

### 7.1 A self-contained primer on reinforcement learning

Reinforcement learning (RL) is how an agent learns to make good decisions by trial and error. The vocabulary is small and worth fixing once, in the terms of our problem:

- **State**  $X_k$ : everything the agent knows when it must decide. Here, a snapshot of every live drone's condition (charge, load, reliability, link rate, tier) at stage  $k$  — where the link rate now reflects the drone's current *position*, so mobility is visible to the policy.
- **Action**  $A_k$ : the decision. Here, the chosen primary-backup pair  $(f_p, f_b)$  for the current task.
- **Policy**  $\pi_\theta$ : the agent's strategy — a function, with tunable parameters  $\theta$ , mapping states to a probability distribution over actions. "Training" means adjusting  $\theta$  so the policy picks better actions.
- **Reward**  $r_k$ : a number scoring how good the action was. The agent is built to maximise the sum of rewards over time, so the reward is where we encode what we want — low delay, balanced load, high reliability, conserved energy.
- **Value**  $V_\phi(X_k)$ : a second network's estimate of the total future reward expected from a state. It is a *critic* that judges how promising a situation is, used to train the policy more stably.

**The learning loop in one breath.** The agent observes the state, samples an action from its policy, receives a reward, and lands in a new state. Over many such steps it adjusts  $\theta$  so that actions which led to high reward become more probable. The actor (policy  $\pi_\theta$ ) proposes actions; the critic (value  $V_\phi$ ) estimates how good states are; together they form the Actor-Critic architecture used here.

## 7.2 Why attention and PPO, not a plain deep Q-network

A common first instinct for a routing problem is a Deep Q-Network (DQN): a fixed-size neural network that scores each possible action. Three properties of this problem make a DQN a poor fit and motivate our choices.

1. **Variable, unordered topology.** Drones die and (in pre-training) rejoin, so the number of nodes changes, and there is no meaningful order among them. A standard network needs a fixed-size, ordered input and breaks when a UAV disappears. Self-attention is *permutation-invariant*: it processes a set of nodes of any size and gives the same answer regardless of the order they are listed in. This is exactly the right inductive bias for a swarm.
2. **Action explosion.** Choosing an ordered pair  $(f_p, f_b)$  from  $N$  drones is one of  $O(N^2)$  actions — hundreds for  $N = 20$ , and the count changes as  $N$  changes. A DQN with one output per action cannot cope. We instead decode the pair *autoregressively*: first pick the primary ( $O(N)$  choices), then the backup conditioned on the primary ( $O(N)$  choices). Two linear steps replace one quadratic one.
3. **Reward heterogeneity.** Our reward mixes quantities in different units (milliseconds, joules, cycles). A DQN must regress the absolute sum of these mixed-unit rewards, which is numerically delicate. PPO (Proximal Policy

Optimisation) optimises a *relative advantage* — whether an action did better or worse than expected — which is far more stable under mixed-unit, shifting rewards.

In short: attention for the set-structured state, autoregressive decoding for the pair action, and PPO for stable learning under a heterogeneous reward.

### 7.3 State and the self-attention encoder

The state is an unordered set of per-drone feature vectors. Each drone contributes five numbers; crucially, the drone’s tier is included as a raw feature so that any tier preference is learned, never hard-wired:

$$x_j^{(k)} = \left[ SOC_j(k), W_{f_j}(k), R_j^0(k), TR_{B,f_j}(k), tier_j \right] \in R^5,$$

where  $tier_j \in \{1, 2\}$  is the emergent tier label of Section 4.2, and  $R_j^0(k)$  is the base reliability of a nominal 200 KB task (at the nominal 1,700 cycles/byte intensity) evaluated at the node’s *current* link rate — so it reflects live link quality and position rather than a fixed distance.

#### 7.3.1 Input normalisation

The five features live on wildly different scales (a charge in  $[0, 1]$ , a workload in tens of millions of cycles, a rate in tens of Mbps). Attention scores are inner products, which are dominated by large-magnitude features unless we rescale. We standardise each feature using running exponential-moving-average statistics  $\mu_{feat}, \sigma_{feat}$  (momentum 0.99, updated only during training rollouts):

$$\tilde{x}_j^{(k)} = \frac{x_j^{(k)} - \mu_{feat}}{\sigma_{feat} + \varepsilon}, \quad \varepsilon = 10^{-5}.$$

#### 7.3.2 Transformer encoder layers

Each normalised vector is first lifted to the model dimension  $d_{model}$  by a linear embedding, then passed through  $L$  stacked Transformer encoder layers. There is no positional encoding — omitting it is what makes the encoder permutation-invariant, treating the drones as a set rather than a sequence:

$$\begin{aligned} e_j^{(0)} &= W_{emb} \tilde{x}_j^{(k)} + b_{emb}, \quad W_{emb} \in R^{d_{model} \times 5}, \\ e_j^{(\ell)} &= LayerNorm\left(e_j^{(\ell-1)} + MHA^{(\ell)}\left(e_j^{(\ell-1)}; \{e_{j'}^{(\ell-1)}\}\right)\right), \\ e_j^{(\ell)} &= LayerNorm\left(e_j^{(\ell)} + FFN^{(\ell)}\left(e_j^{(\ell)}\right)\right), \quad h_j = e_j^{(L)}. \end{aligned}$$

The multi-head attention uses  $n_h$  heads (each of dimension  $d_k = d_{model}/n_h$ ), and the feed-forward block is a two-layer network with hidden width  $d_{ff}$ :

$$head_m = Softmax\left(\frac{Q_m K_m^\top}{\sqrt{d_k}}\right) V_m, \quad FFN(u) = W_2 ReLU(W_1 u + b_1) + b_2.$$

Finally, a single global context vector summarises the whole swarm by mean-pooling the node embeddings:

$$c_k = \frac{1}{|F|} \sum_{j=1}^{|F|} h_j.$$

**What attention actually computes.** Each drone's embedding  $h_j$  is a weighted blend of all drones' features, where the blend weights (the softmax of query-key products) say "how relevant is drone  $j'$  to drone  $j$ ." So  $h_j$  is not just drone  $j$  in isolation — it encodes drone  $j$  in the context of the rest of the swarm. A drone that is the only one with charge left will look very different in its embedding than the same drone in a fully-charged swarm. The context  $c_k$  then gives the policy a global summary to compare any single drone against.

**Output (encoder).** A set of node embeddings  $H_k = \{h_j\}$  and a global context  $c_k$ , each a vector in  $R^{d_{model}}$ . These are the only things the actor and critic see.

## 7.4 Autoregressive action with safety masking

The action is the ordered pair  $(f_p, f_b)$ . We decode it in two steps, each a softmax over the drones, with masks that forbid unsafe choices by setting their score to  $-\infty$  (so their probability becomes exactly zero).

### 7.4.1 Step 1 — choose the primary $f_p$

A small network scores each drone from its embedding and the global context, and the primary mask forbids any drone below 30% charge:

$$z_p^{(j)} = MLP_p([h_j \parallel c_k]), \quad M_p(j) = \{-\infty \text{ if } SOC_j < 0.30, 0 \text{ otherwise}\}$$

$$\pi_\theta(f_p | X_k) = Softmax(z_p + M_p).$$

### 7.4.2 Step 2 — choose the backup $f_b$ , conditioned on the primary

The backup score additionally sees the chosen primary's embedding  $h_{f_p}$ , so the backup is selected *in light of* the primary. The backup mask forbids reusing the primary and forbids any drone below 15% charge:

$$z_b^{(j)} = MLP_b \left( \left[ h_j \parallel c_k \parallel h_{f_p} \right] \right), \quad M_b(j) = \begin{cases} -\infty & j = f_p \text{ or } SOC_j < 0.15 \\ 0 & \text{otherwise} \end{cases}$$

$$\pi_\theta(f_b | X_k, f_p) = Softmax(z_b + M_b).$$

**Design note: the corrected backup mask.** An earlier version of this rule masked out backups with  $SOC \geq 0.30$  — which perversely forced backups onto the most depleted drones, the ones least able to finish the task, defeating the entire purpose of redundancy. The corrected rule requires the backup to retain  $SOC \geq 0.15$ , i.e. enough charge to actually complete the task. Any preference for using a lower-tier drone as the backup is expressed through the learned tier feature, never through charge.

Because the two steps are independent given the state and the primary, the joint action probability is their product:

$$\pi_\theta(A_k | X_k) = \pi_\theta(f_p | X_k) \cdot \pi_\theta(f_b | X_k, f_p).$$

**Why decode in two steps (autoregression).** Rather than score all  $O(N^2)$  pairs at once, we pick the primary, then pick the backup knowing who the primary is. This is the same trick language models use to write a sentence one word at a time: each choice is cheap and informed by the previous. It collapses a quadratic action space into two linear ones and lets the backup choice genuinely depend on the primary.

**Output (action).** The pair  $A_k = (f_p, f_b)$ , together with the two component log-probabilities stored separately — they are needed later, in log-space, to form the PPO ratio without numerical underflow.

## 7.5 Reward (an explicit, unambiguous sign convention)

The reward encodes everything we want. It is built from *improvements*, so that “better” is always a positive number, plus an energy term and a hard safety penalty. First, the predicted energy of assigning  $\tau_i$  to drone  $f_j$  counts the hover energy wasted in the queue, the active execution energy, and the communication energy:

$$E_{consume}(f_j, \tau_i) = \underbrace{(P_{hover} + P_{idle}) T_{f_j}^{Q,avg}}_{\text{wasted queue energy}} + \underbrace{(P_{hover} + P_{proc}) T_{f_j, \tau_i}^e}_{\text{active execution}} + E_{comm,j}.$$

Next, the three improvement deltas are defined so that an improvement is positive regardless of whether the underlying criterion is beneficial or not:

$$\Delta WL = WL_{k-1} - WL_k, \quad \Delta D = D_{k-1} - D_k, \quad \Delta R = R_k - R_{k-1}.$$

The non-beneficial quantities ( $WL, D$ ) reward their decrease; the beneficial quantity ( $R$ , the per-task combined reliability of the dispatched pair) rewards its increase. Each delta is normalised by its own previous value, making the terms dimensionless and comparable despite their different units — and each normalised ratio is **clamped to**  $[-1, 1]$  (with a small  $\varepsilon = 10^{-8}$  guarding the denominator). With all weights positive, the per-step reward is

$$r_k = \rho_1 \text{clip}\left(\frac{\Delta WL}{WL_{k-1}}\right) + \rho_2 \text{clip}\left(\frac{\Delta D}{D_{k-1}}\right) + \rho_3 \text{clip}\left(\frac{\Delta R}{R_{k-1}}\right) + \rho_4 \text{clip}\left(\frac{E_{f_p}^{res}(t) - E_{consume}(f_p, \tau_i)}{E_{initial,p}}\right).$$

Finally, a hard penalty enforces the safety floor: if the primary's charge falls below the 15% Return-to-Home threshold, the composite is overridden entirely:

$$r_k = -M \quad (M = 100).$$

**Design note: why the clamp is essential.** Near episode start the queues are almost empty ( $WL_{k-1} \rightarrow 0$ ) and reliability is high ( $R_{k-1} \rightarrow 1$ ), so an unclamped ratio  $\Delta X/X_{k-1}$  can blow up to  $\sim 10^7$  when the denominator is tiny — a single such spike would dominate the return and destabilise PPO. Clamping keeps every term in a bounded, comparable range (standard reward-shaping practice) without changing the sign convention.

**Why improvements, not absolute values.** If the reward were the absolute delay, the agent could be “rewarded” simply by tasks happening to be easy, and punished for hard tasks outside its control. Rewarding the *change* it caused — did this action reduce delay relative to the last step? — isolates the agent's own contribution and keeps the reward scale stable as conditions drift. The large negative  $M$  makes a battery-depleting choice unambiguously catastrophic, so the policy learns hard to avoid it.

**Output (reward).** A scalar  $r_k$  in which the sign of every term is unambiguous: positive for improvement, strongly negative for a safety violation.

## 7.6 Critic, advantage (GAE), and the value target

The critic estimates the value of a state — the total future reward expected from it — by a small network on the global context:

$$V_{\phi}(X_k) = MLP_v(c_k).$$

To judge a single action we use the *advantage*: how much better the outcome was than the critic expected. The one-step temporal-difference error

$$\delta_k = r_k + \gamma V_{\phi}(X_{k+1}) - V_{\phi}(X_k)$$

compares the realised reward-plus-next-value against the predicted value ( $\gamma$  is the discount that values near-term reward slightly above distant reward). Generalised Advantage Estimation (GAE) sums these errors over the future with an exponential decay  $\lambda$ , trading off bias against variance:

$$\hat{A}_k = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{k+l}.$$

The regression target the critic is trained towards is the advantage plus the (stop-gradient) current value:

$$V_{target}^{(k)} = \hat{A}_k + V_{\phi}(X_k)|_{sg} \quad (sg = stop - gradient).$$

**Advantage = surprise, in a good sense.** A positive advantage means the action turned out better than the critic predicted, so the policy should make it more likely; a negative advantage means worse than predicted, so less likely. GAE's  $\lambda$  knob blends a low-variance one-step estimate ( $\lambda \rightarrow 0$ ) with a high-information full-return estimate ( $\lambda \rightarrow 1$ ); the chosen  $\lambda = 0.95$  sits near the informative end while controlling variance.

**Output (critic/GAE).** The value estimate  $V_{\phi}(X_k)$ , the advantage  $\hat{A}_k$ , and the regression target  $V_{target}^{(k)}$  for the value loss.

## 7.7 The PPO objective (clip + value + entropy)

PPO improves the policy in small, safe steps. It compares the new policy to the old one through a probability ratio, computed in log-space from the two stored component log-probabilities to avoid underflow:

$$r_t(\theta) = \frac{\pi_{\theta}(f_p|X_k)\pi_{\theta}(f_b|X_k,f_p)}{\pi_{\theta_{old}}(f_p|X_k)\pi_{\theta_{old}}(f_b|X_k,f_p)}.$$

The clipped surrogate objective rewards moving probability towards high-advantage actions but refuses to move too far in one update, by clipping the ratio to  $[1 - \varepsilon_{clip}, 1 + \varepsilon_{clip}]$ :

$$L^{CLIP}(\theta) = \hat{E}_k \left[ \min \left( r_t \hat{A}_k, \text{clip} \left( r_t, 1 - \varepsilon_{clip}, 1 + \varepsilon_{clip} \right) \hat{A}_k \right) \right].$$

The critic is trained by squared error against its target:

$$L^{VF}(\phi) = \hat{E}_k \left[ (V_\phi(X_k) - V_{target}^{(k)})^2 \right].$$

An entropy bonus over the non-masked support keeps the policy exploring rather than collapsing prematurely onto one choice — important precisely because the masks already prune the action space hard:

$$H[\pi_\theta] = H[\pi_\theta(f_p | X_k)] + E_{f_p \sim \pi_\theta} [H[\pi_\theta(f_b | X_k, f_p)]].$$

The single loss minimised each epoch combines the three, with coefficients  $c_1, c_2$ :

$$L(\theta, \phi) = \hat{E}_k \left[ -L^{CLIP}(\theta) + c_1 L^{VF}(\phi) - c_2 H[\pi_\theta] \right].$$

Each collected batch receives 4 inner gradient epochs with the Adam optimiser at learning rate  $3 \times 10^{-4}$ , and gradients are clipped to a maximum norm of 0.5 — the standard PPO stabilisation trio.

**Why clipping is the whole idea of PPO.** If the policy chased a high-advantage action with an unbounded step, one lucky sample could push it into a bad region it cannot recover from. Clipping caps how much a single update can change any action's probability, so learning proceeds in small, reliable improvements — “proximal” meaning the new policy stays close to the old one. The minus signs make the maximisation of reward and exploration into a minimisation the optimiser can run; the plus sign on  $c_1 L^{VF}$  means the critic's error is genuinely being reduced.

**Output (PPO).** Updated actor and critic weights  $(\theta, \phi)$  after each training epoch.

## 7.8 Hyperparameters

| Parameter       | Symbol      | Value |
|-----------------|-------------|-------|
| Model dimension | $d_{model}$ | 128   |
| Attention heads | $n_h$       | 8     |

| Parameter                              | Symbol                           | Value                  |
|----------------------------------------|----------------------------------|------------------------|
| Encoder layers                         | $L$                              | 3                      |
| FFN hidden width                       | $d_{ff}$                         | 256                    |
| KL threshold                           | $\theta_{KL}$                    | 0.05                   |
| Reward weights                         | $\rho_1, \rho_2, \rho_3, \rho_4$ | 0.15, 0.35, 0.35, 0.15 |
| Discount                               | $\gamma$                         | 0.99                   |
| GAE parameter                          | $\lambda$                        | 0.95                   |
| PPO clip                               | $\varepsilon_{clip}$             | 0.2                    |
| Value coefficient                      | $c_1$                            | 0.5                    |
| Entropy coefficient                    | $c_2$                            | 0.01                   |
| PPO inner epochs                       | —                                | 4                      |
| Learning rate (Adam)                   | —                                | $3 \times 10^{-4}$     |
| Gradient clip (max norm)               | —                                | 0.5                    |
| Feature-normalisation epsilon          | $\varepsilon$                    | $10^{-5}$              |
| Reward denominator epsilon             | —                                | $10^{-8}$              |
| Critical penalty                       | $M$                              | 100                    |
| Reliability floor                      | $R_{min}$                        | 0.95                   |
| Reliability log clamp (aggregate only) | —                                | $10^{-12}$             |
| Primary eligibility                    | —                                | $SOC \geq 0.30$        |
| Backup eligibility                     | —                                | $SOC \geq 0.15$        |
| Cruise speed (mobility)                | $V_{cruise}$                     | 12 m/s                 |
| Standoff ring radius (mobility)        | —                                | 250 m                  |
| Altitude band (mobility)               | $z_j$                            | 80–150 m               |

The reward weights are worth a remark: delay and reliability ( $\rho_2 = \rho_3 = 0.35$ ) dominate, since missing deadlines and failing tasks are the direct drivers of the drop rate, while load balancing and energy ( $\rho_1 = \rho_4 = 0.15$ ) act as softer regularisers that keep the swarm healthy over the long run.

## 7.9 Offline pre-training (solving cold-start)

A freshly initialised policy is dangerous: while it is still random it would repeatedly trigger the  $-M$  penalty by draining real batteries. We therefore pre-train the actor and critic in a *digital twin* — a simulator driven by historical task traces  $T_{hist}$  — where incurring penalties costs nothing but compute. The live broker then initialises from these pre-trained weights  $(\theta, \phi)$  and restricts online learning to small fine-tuning updates, so the deployed system is competent from its first task and adapts gently thereafter.

## 8 Execution and Drop Policy

The router emits a pair; this stage actually dispatches it and scores the outcome. The dispatch logic is deliberately conservative about the backup, to avoid wasting a second drone's energy unless it is truly needed.

### 8.1 Broker dispatch with delayed backup-dropping

For each task the broker proceeds in four steps:

1. Send  $\tau_i$  to the primary  $f_p$  immediately (the task enters  $Q_{f_p}^P$  at its EDF position; the primary's task energy is charged).
2. **Hold** the backup — do not dispatch it yet.
3. If the primary returns a result before the deadline  $d_i$ , delete the held backup (it was never needed; its energy is saved — the common case, well over 99% of tasks).
4. Otherwise, dispatch to the backup  $f_b$ , but only if the remaining time still covers the backup's own worst-case service time,

$$T_{remaining} \geq T_{f_b, \tau_i}^u + T_{f_b}^{Q, avg} + T_{f_b, \tau_i}^e,$$

where  $T_{remaining} = \max(0, d_i - D_i^{primary})$  and the broker decision overhead is *not* charged again (it was already paid on the primary leg). If the budget no longer covers the backup path, the backup arrives too late and the task is counted as failed. When dispatched, the backup enters  $Q_{f_b}^B$  and only then is the backup's task energy charged.

**Why delay the backup.** Eagerly running both drones would double the energy cost of every task. By holding the backup and releasing it only when the primary is clearly going to miss — and only with just enough time left for the backup to finish — the system pays for redundancy exactly when redundancy is about to be needed, and not a moment sooner.

## 8.2 Node queues

Each drone maintains its two queues,  $Q_j^P$  (primary) and  $Q_j^B$  (backup), each capped by the 64-slot design bound. The backup queue has absolute, non-preemptive priority over the primary queue — a backup is, by the time it is dispatched, a task in danger of being dropped, so it must run first — and within each queue tasks are ordered Earliest-Deadline-First.

## 8.3 Success and drop scoring

Let  $I_{func}(f_x) \in \{0, 1\}$  indicate whether drone  $f_x$  functionally survives long enough to finish, drawn from its same per-node failure rates  $\lambda_x, \mu_x$  used in the reliability model of Section 5.5, over the node's actual exposure window (queue wait plus execution). A task succeeds if either the primary or the backup both survives and meets the deadline:

$$Success_i = \{1 \left( I_{func}(f_p) = 1 \wedge D_i^{primary} \leq d_i \right) \vee \left( I_{func}(f_b) = 1 \wedge D_i^{backup} \leq d_i \right) 0 \text{ otherwise.}$$

The realised per-task reliability recorded for the aggregate is  $R_i = R_{i,f_p}^0$  when the backup was never dispatched, and the parallel combination of Section 5.5 when it was. The episode's drop rate — the headline metric the entire system minimises — is the fraction of tasks that did not succeed:

$$Drop\_Rate = 1 - \frac{1}{|T|} \sum_{i=1}^{|T|} Success_i.$$

**Output (execution).** A per-task indicator  $Success_i \in \{0, 1\}$  and the episode-level  $Drop\_Rate \in [0, 1]$ . The latter is the primary evaluation metric of the whole paper, reported alongside the log-domain system-reliability aggregate and the propulsion-energy totals per movement mode.

## 9 Per-Stage Output Reference (Implementation Checklist)

Table 3 is a single-glance contract for an implementer: what each step reads, what it emits, and the shape or range of that output. If a re-implementation matches this table at every row, the data plumbing is correct.

Table 3: What each step reads and emits.

| Step                  | Reads                                       | Emits                               | Range / shape      |
|-----------------------|---------------------------------------------|-------------------------------------|--------------------|
| §4.2 Tiering          | hardware from $q_j$                         | $CE_j^{eff}, tier_j$                | scalar; $\{1, 2\}$ |
| §4.5 Mobility         | mode, hotspots, stations                    | positions $(x, y, z_j)$ , speed $V$ | m; m/s             |
| §5.1 Delay            | $s_i, c_i, fr_j^{avg}$ , queue, slant range | $D_i$                               | ms, scalar         |
| §5.2 Energy           | $P(V)$ , task set $E_j$                     | $E_{f_j}^{res}, SOC_j$              | ]; $[0, 1]$        |
| §5.3 Memory           | capacities, resident tasks                  | $ME_{f_j}$                          | $[0, 1]$           |
| §5.4 Workload         | queued $l$                                  | $W_{f_j}$ (node), $WL$ (system)     | cycles             |
| §5.5 Reliability      | $\lambda_j, \mu_j$ , exposure               | $R_{i,j}^0, R_i, \log_{10} R_{sys}$ | $[0, 1]; \leq 0$   |
| §6.2–6.3 MEREK/Kalman | criteria matrix                             | $z_k, w_k$                          | $\Delta^4$         |
| §6.4 KL gate          | $w_k, w_{k-1}$                              | run_SPOTIS                          | bool               |
| §6.5 SPOTIS           | $w_k$ , ISP, bounds                         | $f_{opt}^{(k)}, L^{(k)}$            | node, list         |
| §7.3 Encoder          | $X_k$                                       | $H_k, c_k$                          | $R^{d_{model}}$    |
| §7.4 Actor            | $H_k, c_k$ , SOC                            | $A_k = (f_p, f_b)$ , log-probs      | node pair          |
| §7.5 Reward           | deltas, energy                              | $r_k$                               | scalar             |
| §7.6 Critic/GAE       | $r_k, V_\phi$                               | $\hat{A}_k, V_{target}$             | scalar             |
| §7.7 PPO              | trajectories                                | $(\theta, \phi)$                    | weights            |
| §8 Execution          | dispatch                                    | Success, Drop_Rate                  | $\{0, 1\}; [0, 1]$ |

## 10 Discussion: How the Pieces Reinforce One Another

It is worth stepping back to see why the two stages need each other. Stage 1 alone would pick a healthy coordinator but route by fixed rules that cannot anticipate the subtle, shifting trade-offs between a fast-but-draining strong drone and a slow-but-fresh weak drone. Stage 2 alone would learn excellent routing but would have no principled, auditable way to choose which drone should host that policy as conditions change. Composing them gives an auditable election feeding a learned routing policy: cheap statistics where transparency matters, learning where nuance matters.

Four threads run through the whole design and tie it together. **Energy is the master constraint:** it appears as the propulsion drain of the mobility layer (4.5), the SOC masks (7.4), the reward's energy term and  $-M$  penalty (7.5), the Return-to-Home dispatch logic (8), and an MCDM criterion (6) — and the power curve's induced-power dip means the mobility controller can actually *save* energy relative to hovering while simultaneously improving link geometry. **Redundancy without waste** is the second: the primary-backup pair provides the reliability lift quantified in Section 5.5 — now scored by the exact product aggregate, so a single catastrophic failure is never averaged away — while delayed backup-dropping (8) keeps its energy cost near zero in the common case. **Permutation invariance** is the third: the attention encoder lets the same trained policy serve a swarm of any size, gracefully absorbing drone failures that would break a fixed-input network. **Rank-reversal-freeness end to end** is the fourth: SPOTIS's fixed bounds, the fixed-constant ME normalisation, and the emergent-but-envelope-pinned hardware model together guarantee that no drone's evaluation ever depends on which *other* drones happen to be alive.

**Limitations and natural extensions.** The model assumes the broker has accurate, low-latency telemetry from every drone; under heavy packet loss this assumption weakens and a partially-observed formulation would be more faithful. The mobility model is deliberately kinematic — horizontal motion at fixed altitude with ring-and-layer separation — which is faithful to how the reference platforms treat repositioning but abstracts away wind, climb energy, and trajectory optimisation; coupling the router to a trajectory planner is a natural next step. The failure processes are modelled as independent exponentials; correlated failures (e.g. a weather front degrading many links at once) would call for the backup to be chosen for *decorrelation*, not just charge and tier. Finally, the policy is trained centrally at the broker; a fully decentralised variant, in which drones negotiate assignments peer-to-peer, is a natural and more fault-tolerant successor.

## 11 Conclusion

We have specified, end to end and from first principles, an energy-aware task distribution system for *mobile* UAV fog networks. A demand-following mobility layer repositions the swarm over its workload hotspots on a collision-free ring-and-altitude scheme, pricing flight and hover on the same rotary-wing power curve — and, thanks to the induced-power dip, sometimes flying for less than hovering costs. The physical models convert raw

configuration, live telemetry, and live geometry into interpretable per-drone quantities, every constant of which now carries an explicit provenance: verified against a reference paper, grounded in standard literature, or declared a documented assumption. Hardware determines capability rather than the reverse: each drone's intrinsic specs yield its computational efficiency, and tiers emerge from the resulting ranking. A transparent four-step MCDM pipeline (MERECEC → Kalman → KL-gate → SPOTIS) elects a Master Fog Head responsively yet cheaply, over a criteria set whose capability member — memory efficiency — is both live and rank-reversal-safe. That broker runs an attention-based PPO router that decodes a fault-tolerant primary-backup pair for each task, trained on a clamped improvement reward guarded by hard energy penalties and warm-started in a digital twin to avoid a dangerous cold start; the resulting reliability is scored by the exact product aggregate, accumulated in log space so it can neither underflow nor forgive a catastrophic failure. Every step's input, output, range, and rationale has been made explicit, and every design choice — attention over fixed inputs, autoregression over a quadratic action head, PPO over Q-learning, improvement rewards over absolute ones, the corrected backup mask, mobility as a first-class model rather than a bolted-on cost — has been motivated by the specific failure it averts. The result is intended to be not merely described but re-implementable, by a reader who began with no background in fog computing, multi-criteria decision-making, or reinforcement learning.